



Take-Home Assignment: Weather-Advisory Support Bot

The problem we're trying to solve

We want an assistant that answers questions about outdoor activity safety ("is it safe to cycle today," "should I take my kid to the park," "is this a good day for a picnic") using live weather data.

Here's a real example, not a hypothetical. This week the India Meteorological Department flagged a well-marked low-pressure area over northeast Madhya Pradesh, bringing heavy to very heavy rainfall through September 5 (isolated spots saw extremely heavy rain on September 3). Separately, squally winds of 45 to 55 km/h, gusting to 65 km/h, are hitting the Bay of Bengal off the south Tamil Nadu coast. Someone in Bhopal asking "is it safe to bike to work today?" isn't asking a theoretical question. There's an actual IMD-flagged rain system over their state right now, with real rainfall numbers attached to it. If our bot answers with a generic "cycling is usually low-risk" because it didn't check, or because nobody thought to write that SOP, a real person makes a real decision on bad advice from us.

We cannot let the assistant make up its own safety advice. We're a business that has to stand behind whatever this bot tells a user. If it tells someone "you're fine to go hiking" and it's wrong, that's on us, legally and reputationally. So every piece of advice has to come from a written policy we control, not from the model's judgment.

Concretely:

- We write a set of rules, we're calling them **Standard Operating Procedures (SOPs)**, that say: under these conditions, give this advice.
- The bot's job is to figure out which rule applies and answer accordingly. It is not the bot's job to decide what good advice is.
- If a user asks something no rule covers, we'd rather the bot say "we don't have guidance for that" than guess. A wrong guess is worse than an honest "I don't know."
- These rules will change over time. The team maintaining them tomorrow should be able to update a policy without touching the code that runs it.

How you structure the rules, how you decide what "matches," how you wire up fetching data, deciding, and answering, is up to you. That's what we're actually assessing.

What you're building

A chat bot, backed by a **LangGraph agent**, that:

1. Takes a user's question about outdoor activity safety.
2. Pulls live weather data relevant to that question.
3. Finds the SOP that applies (or determines none does).
4. Replies with advice that's traceable back to that SOP, never a free-floating answer that just sounds reasonable.

LangGraph is a hard requirement. We want a real graph with branching, not a single prompt-and-response chain wearing a LangGraph label. Beyond that, the internal structure (how many nodes, what they're called, where the branches sit) is your call. We do care that you can defend it.

A few things worth spelling out up front:

- **Model choice is yours.** Use whatever LLM you have API access to (OpenAI, Anthropic, whatever). We're not testing which one you pick, just how you constrain it. Keep your API key out of git. A `.env` file plus a `.gitignore` entry is enough, don't make us find a key in your commit history.
- **Treat this as a session, not isolated one-offs.** Within a single chat session the bot should remember earlier turns. If a user already asked about cycling in Bhopal and got an answer grounded in today's rain, a follow-up like "what about this evening instead?" should build on that context instead of starting from zero. You decide how much state to carry (raw message history, a summarized decision log, structured facts pulled from earlier turns, your call), but the bot shouldn't make the user repeat themselves or contradict what it already said earlier in the same session. Memory resets between sessions; we're not asking for persistence across restarts or across users.
- **Budget your time.** This should take a focused person about a day, not a week. We'd rather see a smaller system where every decision is deliberate than a sprawling one that's half-explained. If you're spending a lot more than that, simplify rather than keep building.

Chat frontend

Don't skip this. A graph with no way to talk to it isn't something we can review. **You need a minimal chat frontend where** a person can type a question and see the bot's reply in a conversational thread. Streamlit, plain HTML and JS, React, a terminal-styled page, your call, whatever you're fastest in. We're not grading design or polish. We're grading whether it runs, whether a reviewer can type into it, and whether a reply comes back. Spend your time on the graph and the SOPs, not the CSS.

Live weather data

Use **Open-Meteo** (<https://api.open-meteo.com/v1/forecast>), free, no API key required.

You must explicitly pass latitude, longitude, and a `current=` (or `hourly=`/`daily=`) field list, or you'll get metadata with no actual weather values and think the API is broken.

Working example:

https://api.open-meteo.com/v1/forecast?latitude=52.52&longitude=13.41¤t=temperature_2m,wind_speed_10m,precipitation,precipitation_probability,uv_index

returns

None

```
{"current": {"time": "2026-09-04T07:30", "temperature_2m": 16.1, "wind_speed_10m": 7.2, "precipitation": 0.0}}
```

For city names, resolve via the free geocoding endpoint first:

<https://geocoding-api.open-meteo.com/v1/search?name=<city>>, take the first result's latitude and longitude, then call `/v1/forecast`.

Fields like `uv_index` must be listed explicitly in `current=`/`hourly=` or they won't show up in the response.

City names aren't always unique. Geocoding "Springfield" or "Bhopal" alongside a same-named town elsewhere can return several candidates. Picking the first result silently is a reasonable default, but if the geocoding call returns nothing at all, or errors out, that's the same "can't resolve a location" failure as the weather API being down. It should route to the same honest fallback, not a half-answer.

The policy rules (SOPs)

An SOP is a written rule: under this condition, give this guidance, at this severity. A few examples of the shape we mean, write your own, don't copy these:

- "If the UV index is 8 or higher between 11am and 4pm, advise against unprotected outdoor exercise during that window and recommend sunscreen plus rescheduling to early morning or evening."
- "If precipitation probability is 70% or higher and the user is asking about travel, warn of possible delays and recommend checking for alerts before departure."
- "If wind speed exceeds 40 km/h and the user mentions cycling or a two-wheeler, flag it as a safety risk, not just a comfort issue."

- "If a well-marked low-pressure system or cyclonic circulation is actively bringing heavy to very heavy rainfall to the user's location, treat every outdoor-activity question as high severity regardless of category, and lead with the rain system itself before any activity-specific advice."

That last one is the case we care about most. A rain system like the one currently sitting over Madhya Pradesh doesn't neatly fall into "outdoor exercise" or "travel," it can touch all of them at once, and the underlying numbers (rainfall totals, wind gusts) may not look extreme in isolation even while the situational risk clearly is. Think about how your matching approach handles a case like that, where the reason is bigger than any single threshold.

Requirements for the rule set:

- At least **10 SOPs**, across at least **3 categories** (outdoor exercise, travel, vulnerable groups like elderly, children, or pets are just examples, your own categories are fine).
- A range of severities. Not everything can be "this is dangerous."
- At least **one SOP for a fuzzy, non-numeric scenario**, something like "is today good for a picnic," where there's no clean threshold to check against. We want to see how you handle advice that doesn't reduce to $\text{if } x > y$.

Once you have 10+ SOPs, sometimes more than one will genuinely apply to the same question (high UV and strong wind on the same cycling question, say). We're not telling you how to resolve that. Pick one and answer with only that SOP, surface both, rank by severity, whatever you decide, but decide on purpose and say what you chose and why. "It just does something" isn't good enough for us to trust in front of a user.

Non-negotiables

These are outcomes we need, however you get there:

- **Every answer must be traceable to a specific SOP**, or the bot must explicitly say no SOP applies. We should always be able to ask "why did it say that" and get a policy citation back, not a shrug.
- **Adding or changing a policy must not require touching the code that fetches weather or calls the model.** If your design fails this test, say so honestly in your write-up rather than hiding it. We'd rather know now.
- **The bot must never answer with a forecast it doesn't actually have.** If the weather API fails or a location can't be resolved, it must say so plainly, not produce a plausible-sounding guess.
- **The bot must never invent generic advice when no policy covers the question.** Saying "I don't have guidance for that" is a correct, acceptable answer.
- **The bot only composes language, it doesn't get to decide facts.** Numbers it reports back (temperature, wind speed, rainfall, whatever) must be the numbers

that actually came from the API for that request, not something the model recalls or estimates. If you can't point to where in your code that's enforced, that's a gap, not a detail.

- We'll ask you, live in the review call, to **add an 11th SOP on the spot** without touching your control-flow code. Design for that moment.

Proving it works

Manual testing only isn't enough for production. Write an eval suite (a script or notebook is fine) covering all cases:

- At least 2 cases where an SOP clearly applies and the answer must reflect it correctly.
- At least 2 cases are phrased so they **don't reuse your SOP's wording**, paraphrased intent, not keywords. We want to know the matching isn't just string lookup.
- At least 1 case with genuinely severe live weather conditions, where the answer must be grounded in the real numbers pulled from the API, not a generic warning. This is a good place to use a real, currently active weather event instead of a fabricated one. At the time of writing, the IMD has flagged a well-marked low-pressure area over northeast Madhya Pradesh bringing heavy to very heavy rainfall through September 5, while squally winds of 45 to 55 km/h, gusting to 65 km/h, are affecting the Bay of Bengal off the south Tamil Nadu coast. Asking "is it safe to go for a bike ride in Bhopal today?" against live Open-Meteo data for those coordinates should pull real, elevated precipitation numbers and produce an answer that cites them and names the SOP that applies, not a canned "rain can be dangerous" line. Whatever event is active when you actually build this will be different from this Madhya Pradesh system by the time we review it, that's fine and expected. Just don't hardcode this event or today's numbers into your logic; the test should hold up against whatever the API returns on any given day.
- At least 1 case where no SOP applies. The bot should say so, kindly, not invent advice.
- At least 1 case simulating an unreachable weather API. The bot must fail honestly.
- At least 1 adversarial case of your own choosing. Think about where a system like this is most likely to break, and go test that. One category worth considering: a user's question is just text that flows into an LLM call. What happens if that text tries to talk the model out of following its SOPs, or tries to get it to claim a policy exists that doesn't? You don't have to pick this one, but if you don't, tell us what you picked instead and why you think it's the more important risk.

For each case: state what you're checking, what a pass looks like, and whether it passed when you ran it. If something fails, say so and explain why. We'd rather see honest gaps than a suite that was guaranteed to pass.

One more wrinkle worth planning for: the Madhya Pradesh low-pressure system is forecast by the IMD to weaken and move on by September 5, and India's monsoon systems shift constantly through the season. Live weather doesn't sit still for your test suite. If your "severe conditions" eval case only passes because you happened to run it during a specific rain event, say so in your write-up. What would you do differently for a suite that needs to keep working after the system passes?

What we're evaluating

- **Policy judgment.** Are your SOPs specific enough to actually be checked against, or vague restatements of "be careful"? Does your approach survive us adding a rule live, without code changes?
- **Architecture.** A real graph with real branching for the failure path, or one function dressed up as a graph?
- **Eval quality.** This carries a lot of weight. Do your test cases actually probe the ways this kind of system breaks (paraphrase robustness, honest "no match," grounding in live data, graceful failure), or are they easy cases picked to pass?
- **Your ability to explain every decision.** How you represented policies, where you drew the boundaries between components, and why you decided what should be deterministic code versus what should be left to the model.

What to hand in

- A standalone git repo.
- A README with setup and run instructions for both backend and frontend.
- Your SOPs, in whatever form you chose, with a one-line note on why you chose that form.
- The LangGraph implementation.
- Your eval suite, with results and honest notes on any failures.